



CS 498: Multi-Dimensional Parallelism

Fan Lai

The Grainger College of Engineering

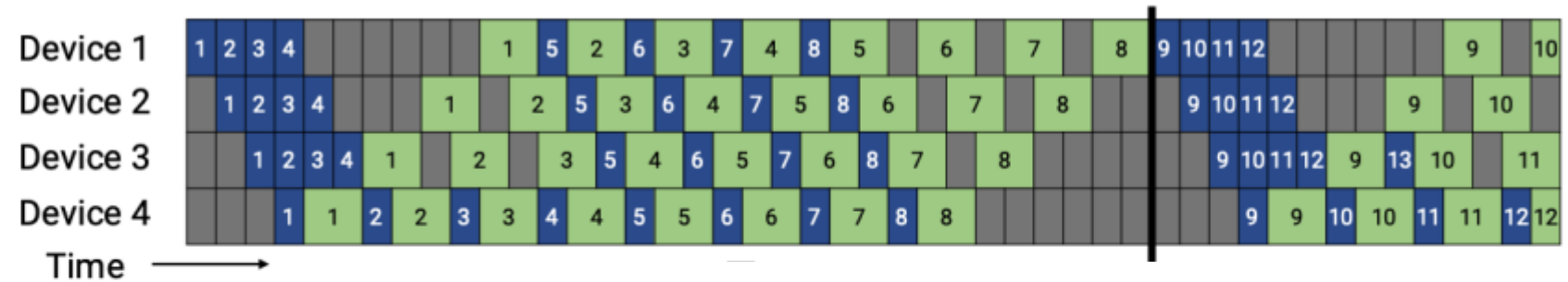
Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

Recall: Pipeline Parallelism with 1F1B Schedule



Pipeline parallelism with 1F1B Schedule

$$\text{BubbleFraction} = \frac{p-1}{m}$$



Recall: Pipeline Parallelism with 1F1B Schedule



Pipeline parallelism with 1F1B Schedule

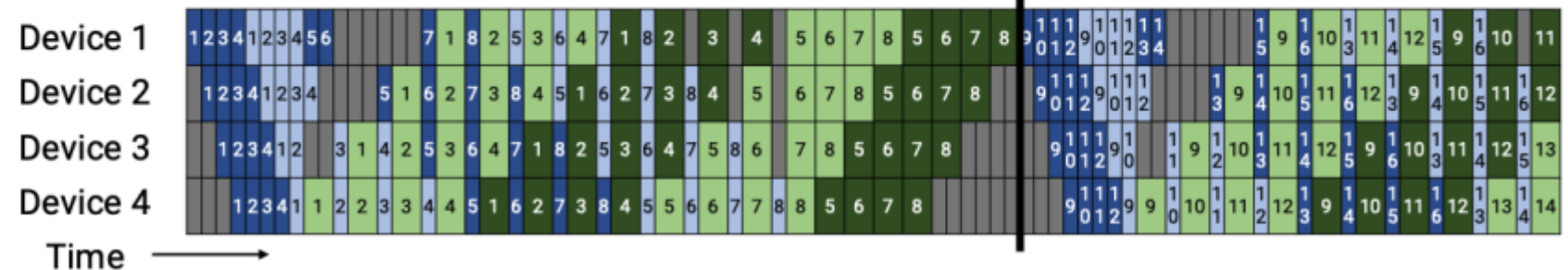
$$\text{BubbleFraction} = \frac{p-1}{m}$$



Assign multiple stages to each device

Pipeline parallelism with interleaved 1F1B Schedule

$$\text{BubbleFraction} = \frac{1}{v} * \frac{p-1}{m}$$



Forward Pass



Backward Pass

Combining Multiple Parallelism Schemes

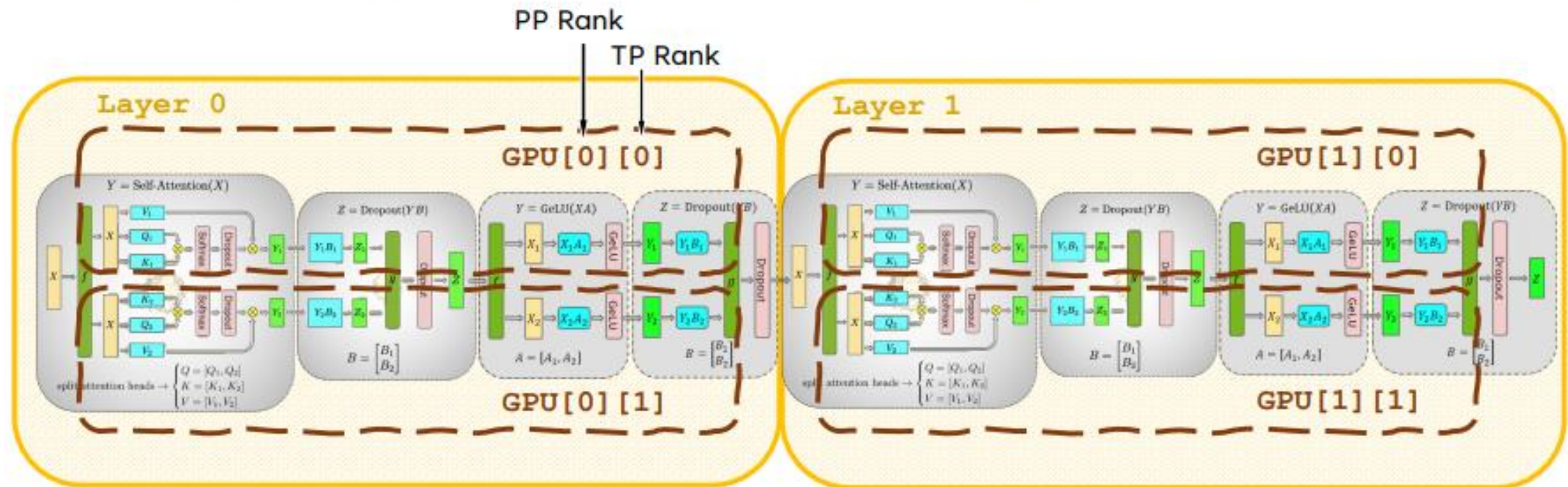
- Data Parallelism (DP), Tensor Parallelism (TP), Pipeline Parallelism (PP)

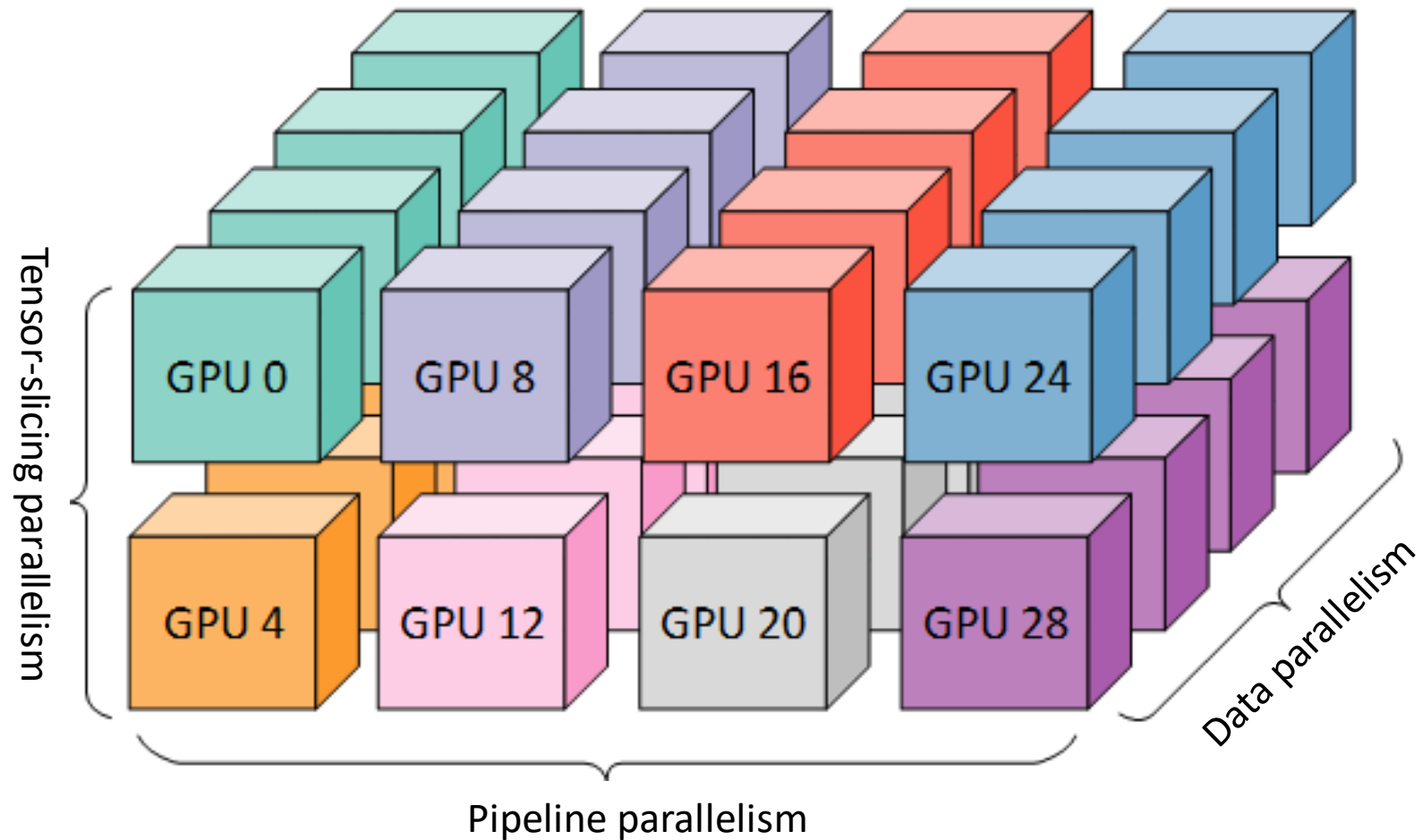
Panel Discussion on Alpa (for automatic model parallelism)

Combining Model Parallelism



- Model_Parallelism = Tensor_Parallelism $\times$ Pipeline_Parallelism





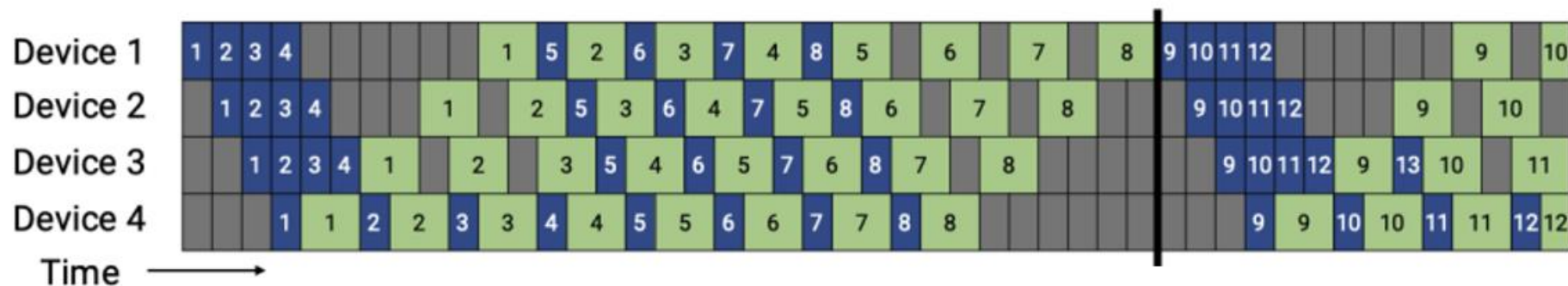
Question: How do we know which parallelism to choose?

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

Tensor and Pipeline Model Parallelism

$$\frac{p-1}{m} \quad (\text{Bubble Fraction})$$

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.



- Tensor and Pipeline Model Parallelism

Assume $d = 1$, $n = p \cdot t$

$$\frac{p - 1}{m} = \frac{n/t - 1}{m}$$

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

- Tensor and Pipeline Model Parallelism

- $t \uparrow$, pipeline bubble $\downarrow$

$$\frac{p-1}{m} = \frac{n/t-1}{m}$$

- Communication overhead

- All-reduce communication for tensor model parallelism is expensive!
- Especially when cross servers

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

- Tensor and Pipeline Model Parallelism

- $t \uparrow$, pipeline bubble $\downarrow$

$$\frac{p-1}{m} = \frac{n/t-1}{m}$$

- Communication overhead

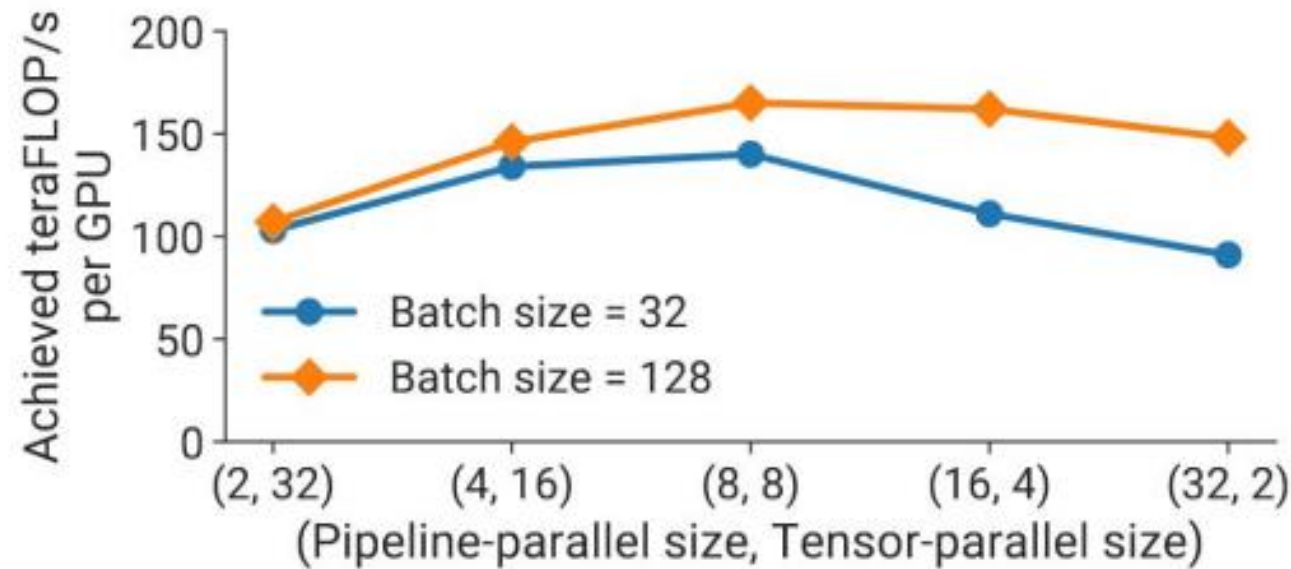
- All-reduce communication for tensor model parallelism is expensive!
- Especially when cross servers

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

Takeaway #1: Use tensor model parallelism within a server and pipeline model parallelism to scale to multiple servers.

- Tensor versus Pipeline Parallelism
 - 161-billion param. GPT

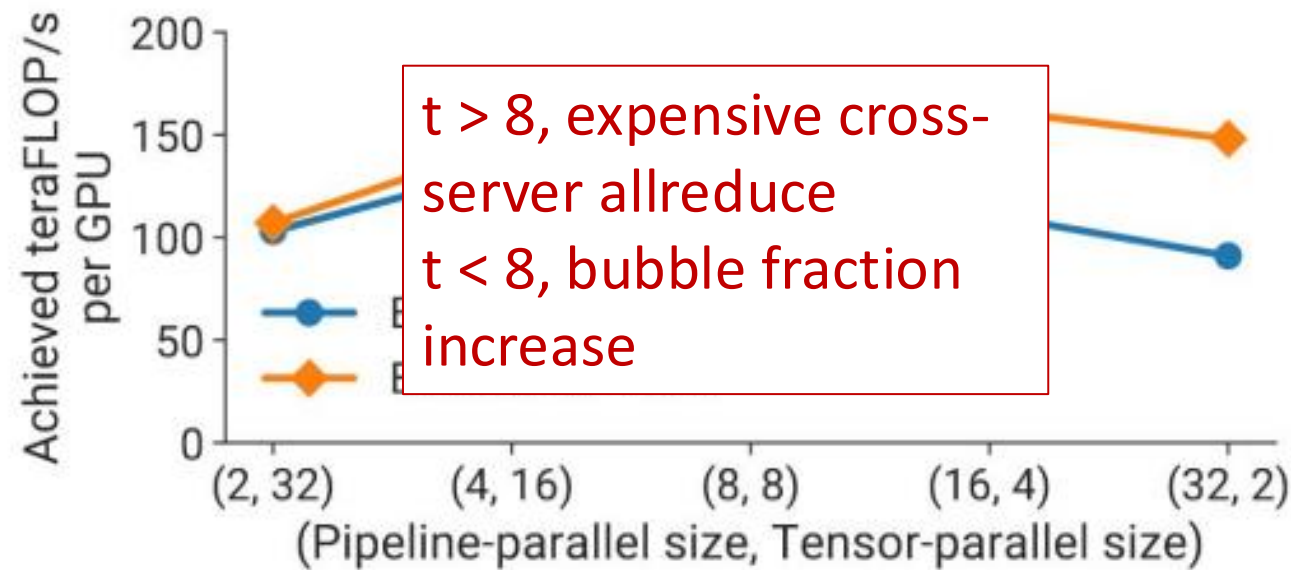
Question: Why does the performance peak at $t = p = 8$?



8 Nvidia 80GB A100 cards per node, 8 nodes are connected through fat-tree topology

- Tensor versus Pipeline Parallelism
 - 161-billion param. GPT

Question: Why does the performance peak at $t = p = 8$?



8 Nvidia 80GB A100 cards per node, 8 nodes are connected through fat-tree topology

- Data versus Pipeline Parallelism

$$\frac{p-1}{m}$$

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

- Data versus Pipeline Parallelism

$$\frac{p-1}{m} = \frac{n/d-1}{B/(d*b)} = \frac{n-d}{B/b}$$

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

- Data versus Pipeline Parallelism

$$\frac{p-1}{m} = \frac{n/d-1}{B/(d*b)} = \frac{n-d}{B/b}$$

- Data versus Tensor Parallelism

- DP is less communication heavy than TP
 - All-reduce once per batch vs. All-reduce once per microbatch
- Tensor parallelism can lead to hardware underutilization

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

- Data versus Pipeline Parallelism

$$\frac{p - 1}{m} = \frac{n/d - 1}{B / (d * b)} = \frac{n - d}{B / b}$$

- (p, t, d) : Parallelization dimensions, where p is the pipeline-model-parallel size, t is the tensor-model-parallel size, and d is the data-parallel size.
- n : Number of GPUs, satisfying $p \cdot t \cdot d = n$.
- B : Global batch size.
- b : Microbatch size.
- $m = \frac{B}{b \cdot d}$: Number of microbatches per pipeline.

- Data versus Tensor Parallelism

- DP is less communication heavy than TP
 - All-reduce once per batch vs. All-reduce once per microbatch
- Tensor parallelism can lead to hardware underutilization

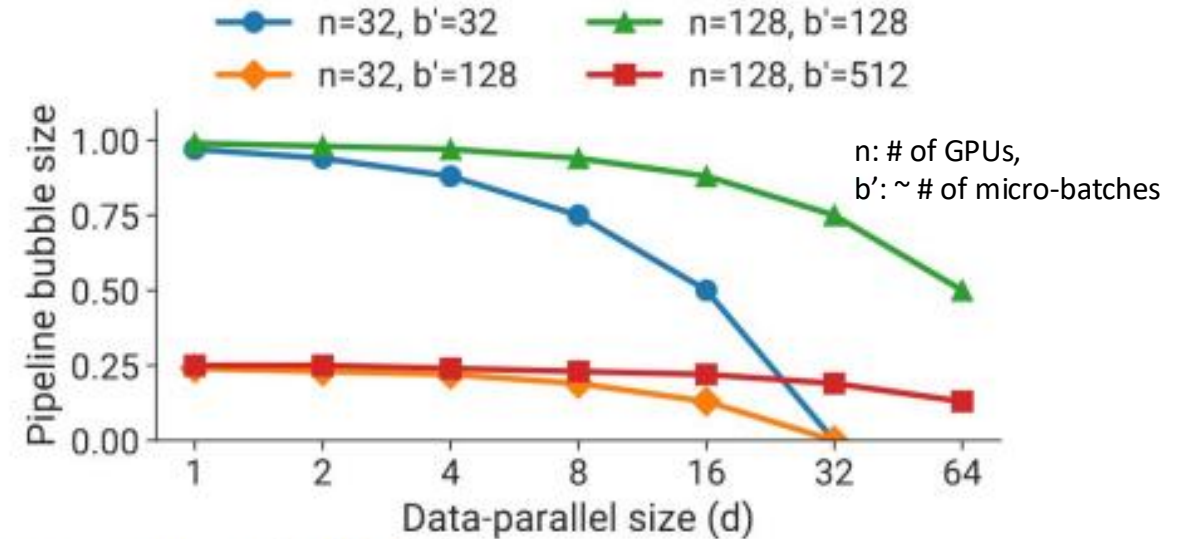
Takeaway #2: Decide tensor-parallel size and pipeline-parallel size based on the GPU memory size; data parallelism can be used to scale to more GPUs.

- Data versus Pipeline Parallelism

$$\frac{p-1}{m} = \frac{n/d-1}{B/(d*b)} = \frac{n-d}{b'=B/b}$$

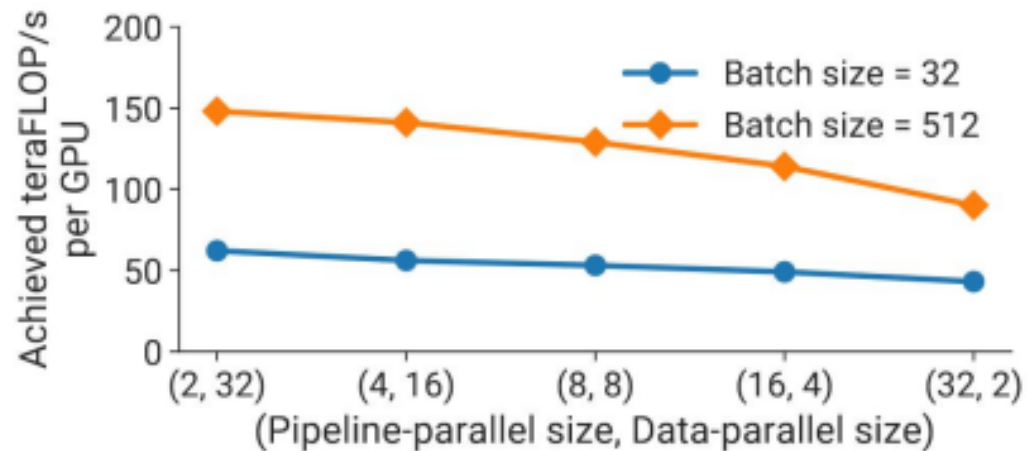
- Data versus Tensor Parallelism

- DP is less communication heavy than TP
 - All-reduce once per batch vs. All-reduce once per microbatch
- Tensor parallelism can lead to hardware underutilization



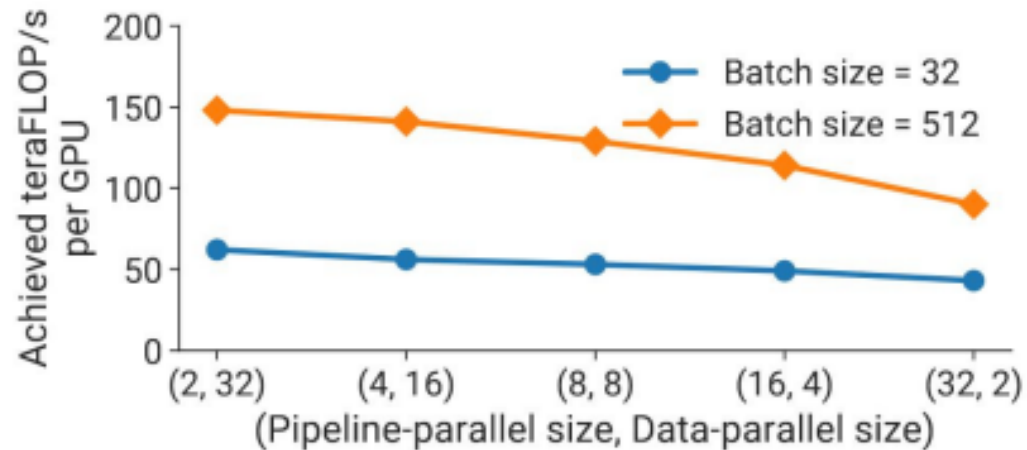
Takeaway #2: Decide tensor-parallel size and pipeline-parallel size based on the GPU memory size; data parallelism can be used to scale to more GPUs.

- Pipeline-parallelism vs. Data-parallelism
 - 5.9-billion param. GPT
 - Throughput decreases as pipeline-parallel size increases



$$\frac{p-1}{m} = \frac{n/d-1}{b'/d} = \frac{n-d}{b'=B/b}$$

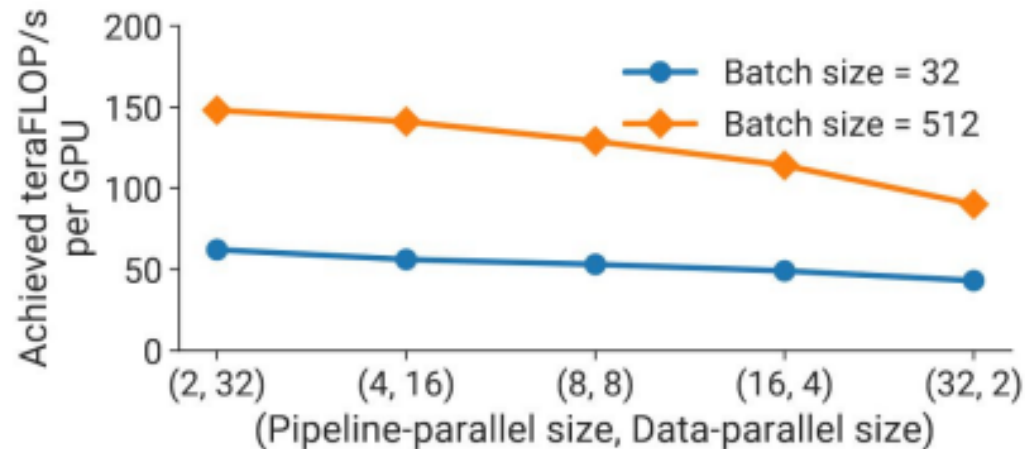
- Pipeline-parallelism vs. Data-parallelism
 - 5.9-billion param. GPT
 - Throughput decreases as pipeline-parallel size increases



Limitations of data-parallelism:

1. Memory capacity
2. Scaling limitation proportional to the batch size

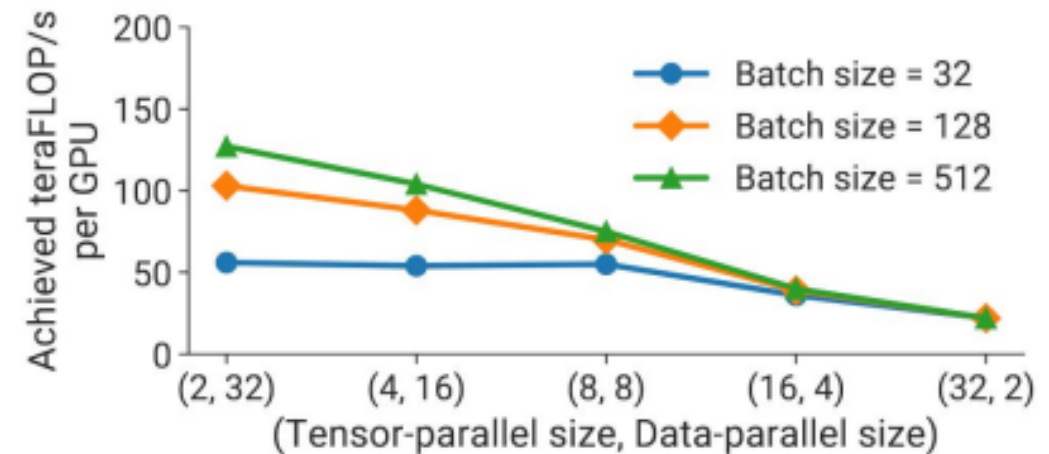
- Pipeline-parallelism vs. Data-parallelism
 - 5.9-billion param. GPT
 - Throughput decreases as pipeline-parallel size increases



Limitations of data-parallelism:

1. Memory capacity
2. Scaling limitation proportional to the batch size

- Tensor-parallelism vs. Data-parallelism
 - 5.9-billion param. GPT
 - Throughput decreases as tensor-parallel size increases

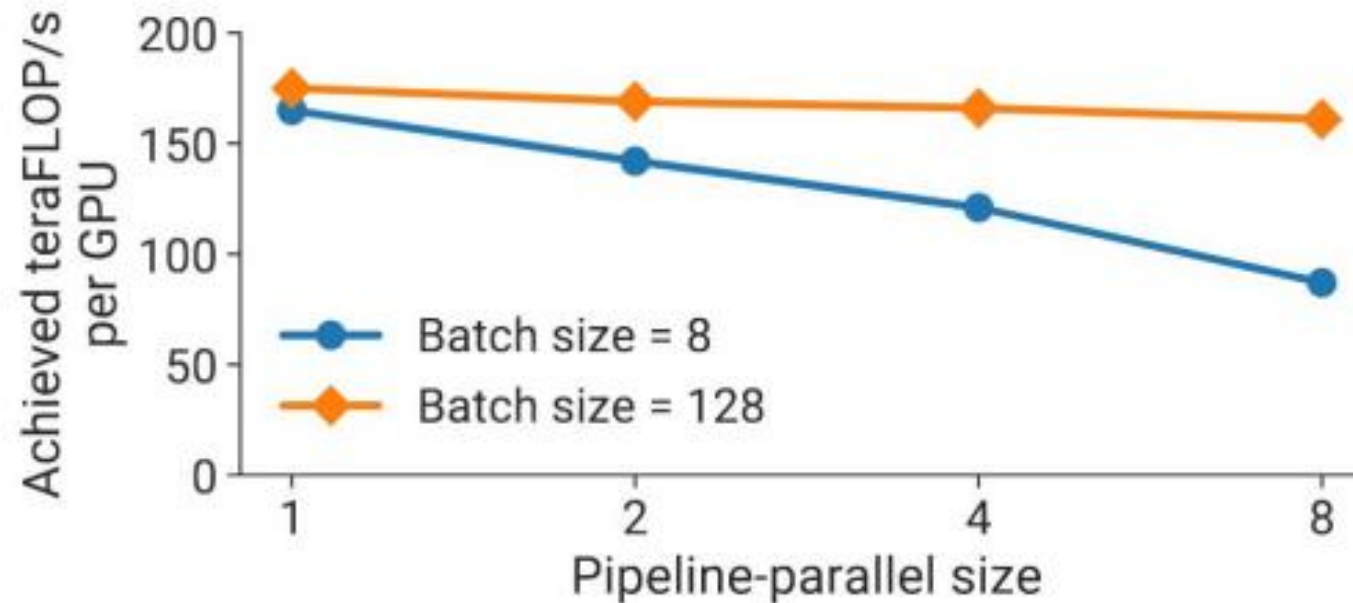


Limitations of tensor-parallelism:

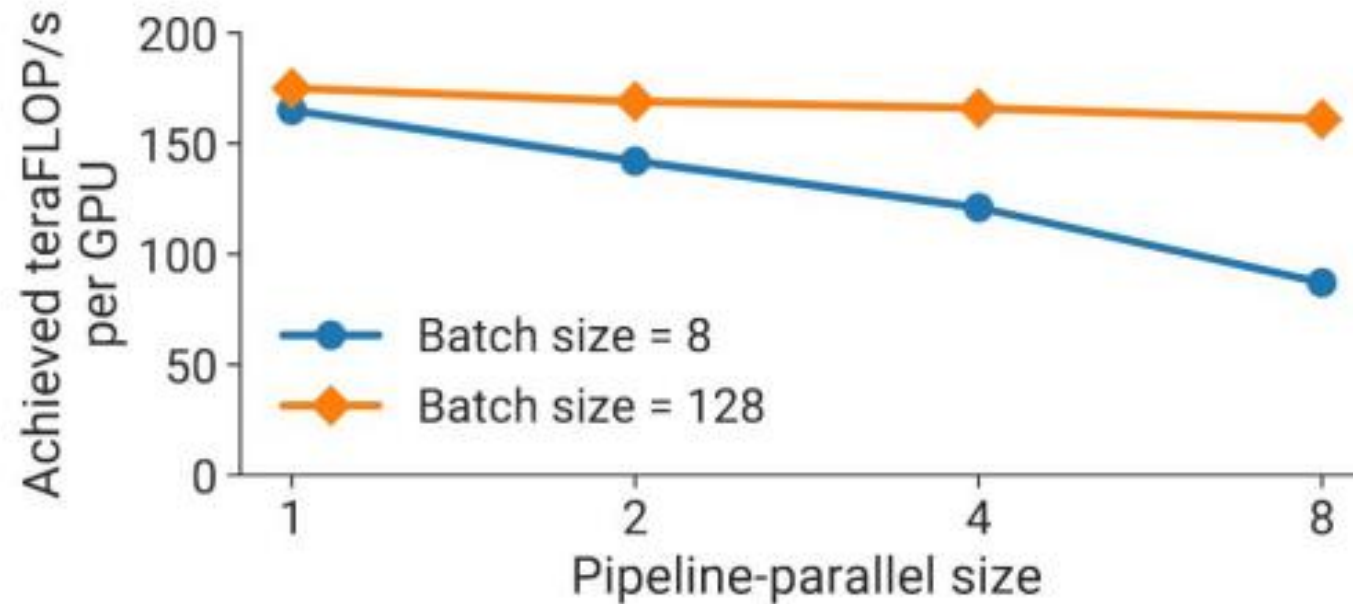
1. More frequent Allreduce
2. Allreduce is on critical path

- Weak Scaling - increase the #layers while increasing PP size

Question: Why does larger batch size scale better?

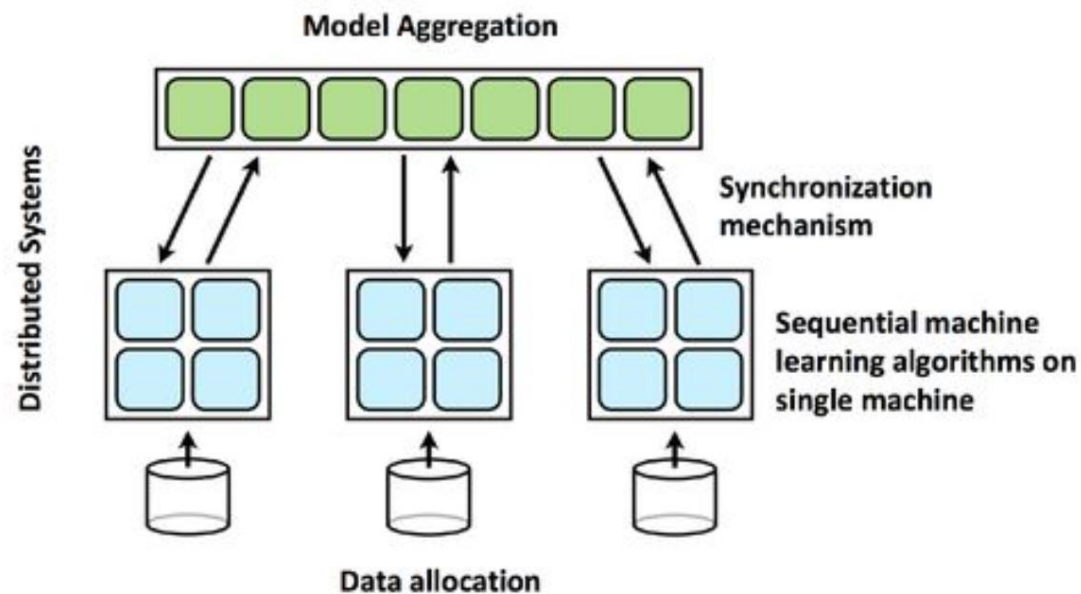


- Weak Scaling - increase the #layers while increasing PP size
- Higher batch size scales better $(p-1)/m$

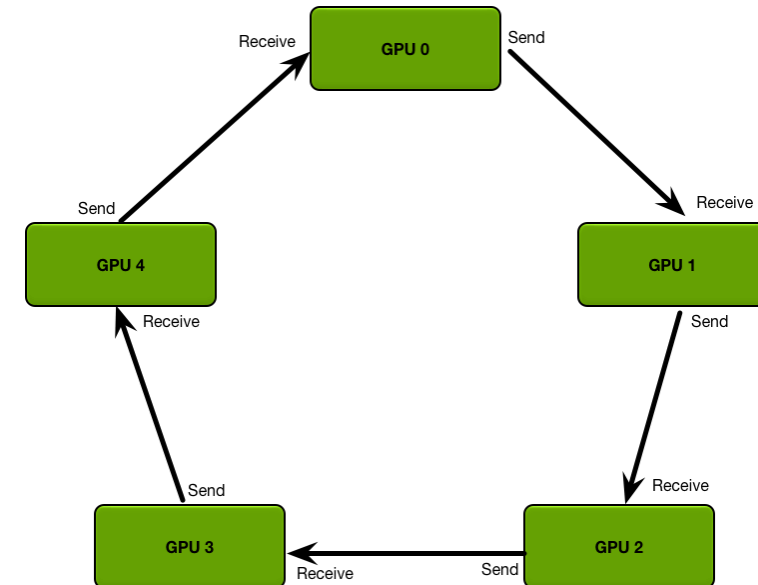


Model Parallelism:

- Data parallelism: cannot run if layer (model) size exceeds GPU capacity



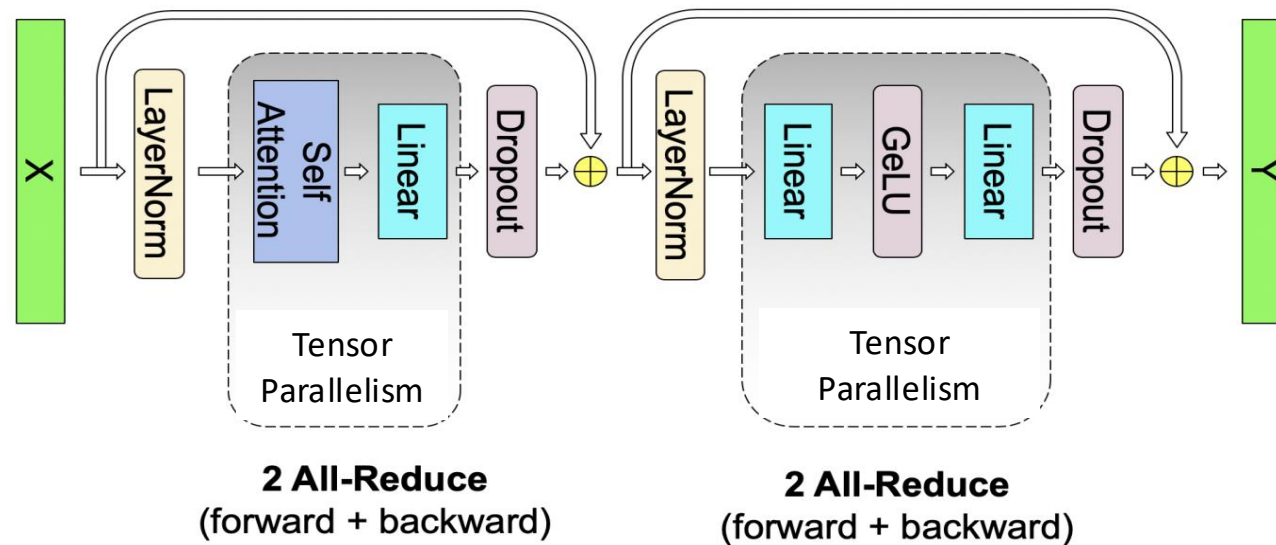
Overwhelm the centralized server



Amortize aggregation but face stragglers

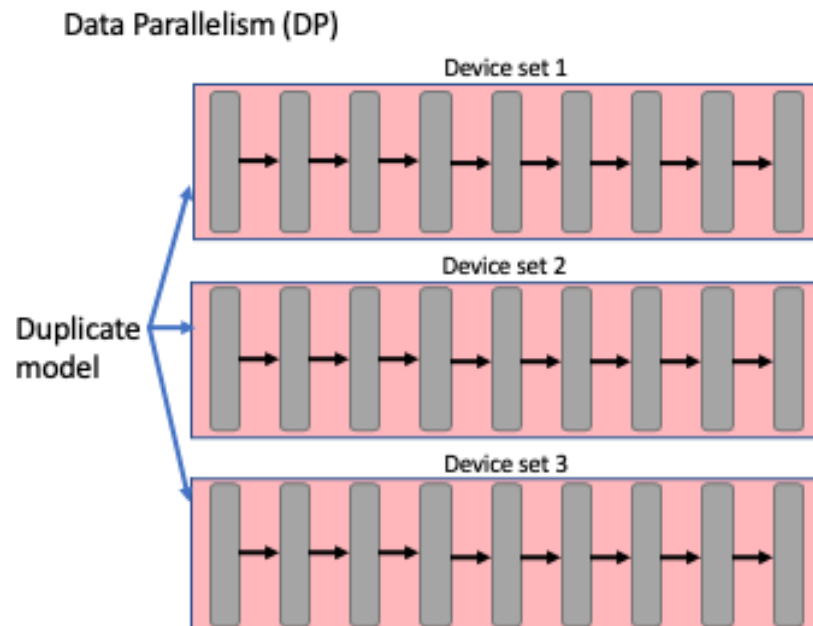
Model Parallelism:

- Data parallelism: cannot run if layer (model) size exceeds GPU capacity
- Tensor parallelism: communication-heavy

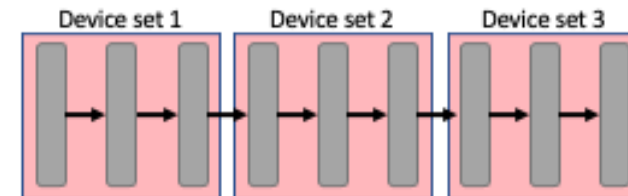


Model Parallelism:

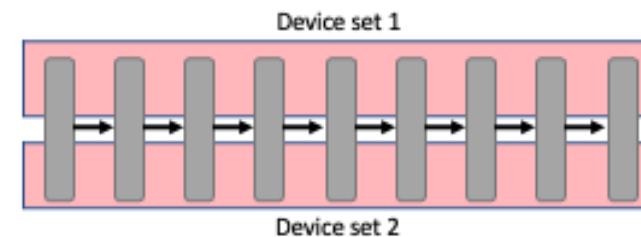
- Data parallelism: cannot run if layer (model) size exceeds GPU capacity
- Tensor parallelism: communication-heavy
- Pipeline parallelism: tuning microbatch sizes to minimize bubbles



Pipeline Parallelism (PP)



Tensor Parallelism (TP)



Model Parallelism:

- Data parallelism: cannot run if layer (model) size exceeds GPU capacity
- Tensor parallelism: communication-heavy
- Pipeline parallelism: tuning microbatch sizes to minimize bubbles

Rule of thumb:

- Use tensor parallelism within a node and pipeline model parallelism to scale to multiple nodes
- Decide tensor-parallel size and pipeline-parallel size based on GPU memory size
- Data parallelism can be used to scale to more GPUs

Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning

Lianmin Zheng^{1,*} Zhuohan Li^{1,*} Hao Zhang^{1,*} Yonghao Zhuang⁴
Zhifeng Chen³ Yanping Huang³ Yida Wang² Yuanzhong Xu³ Danyang Zhuo⁶ Eric P. Xing⁵
Joseph E. Gonzalez¹ Ion Stoica¹

¹UC Berkeley ²Amazon Web Services ³Google ⁴Shanghai Jiao Tong University
⁵MBZUAI, Carnegie Mellon University ⁶Duke University

Introduce how to automatically decide model parallelism for an ML training job

- Inter-operator design: dynamic programming to minimize end-to-end training latency
- Intra-operator design: linear programming to decide tensor-GPU mappings

Assumes **homogeneous** GPUs!